While we start with an interval of 500 milliseconds for the first time through our code (inside the `if` statement), we have added this line at the bottom:

```
interval = random(250, 1001);
```

This will generate a random delay between a quarter second and 1 second by assigning a random value between 250 and 1000 to the variable interval, even though we do not always need to assign the random value to a variable for it to work. Instead, because the function works in a similar way as `millis()` in that it returns a value to the place in the program where it was called, we can use it in place of a variable. Take the following code fragment:

```
for (int i=0; i<=255; i+=5) {
  analogWrite(5, i);
  delay(random(50));
}
for (int i=255; i>=0; i-=5) {
  analogWrite(5, i);
  delay(random(50));
}
```

In this sample code we fade an LED connected to pin 5 up to full brightness and back to off again with a random delay between each step in brightness. By placing the `random()` function with a specified maximum of 50 inside of the `delay()` function, each iteration through the `for` loop will pause for a random time period between 0 and 49 milliseconds.

Generating random numbers in our sketches has so many possible uses that we are only scratching the surface here. One last possibility before we look at how we can make our semi-random numbers a little better, is using the random function to choose one of several possible outcomes using a `switch` statement. Take this modified code fragment borrowed from the last chapter:

```
randomNumber = random(1,4);
  switch (randomNumber) {
    case 1:
    ;
    case 2:
    ;
    case 3:
    ;
  }
```

In this fragment, we generate a random number that includes the possible values 1, 2, and 3 and assign that value to the variable `randomNumber`. Referencing this variable in the `switch` statement will allow for one of three randomly chosen possible outcomes to be chosen depending on the value that was generated.

randomSeed()

Now because our random numbers are only semi-random, the `random()` function doesn't produce the most random numbers—depending, of course, on how you look at it. That's because the Arduino microcontroller—being a stubborn little computer that is fairly deterministic—uses a set formula to create a sequence of values that, while a fairly long sequence, only *appears* random and will inevitably repeat. It also means that each time you power on the Arduino interface board or give it a reset, the